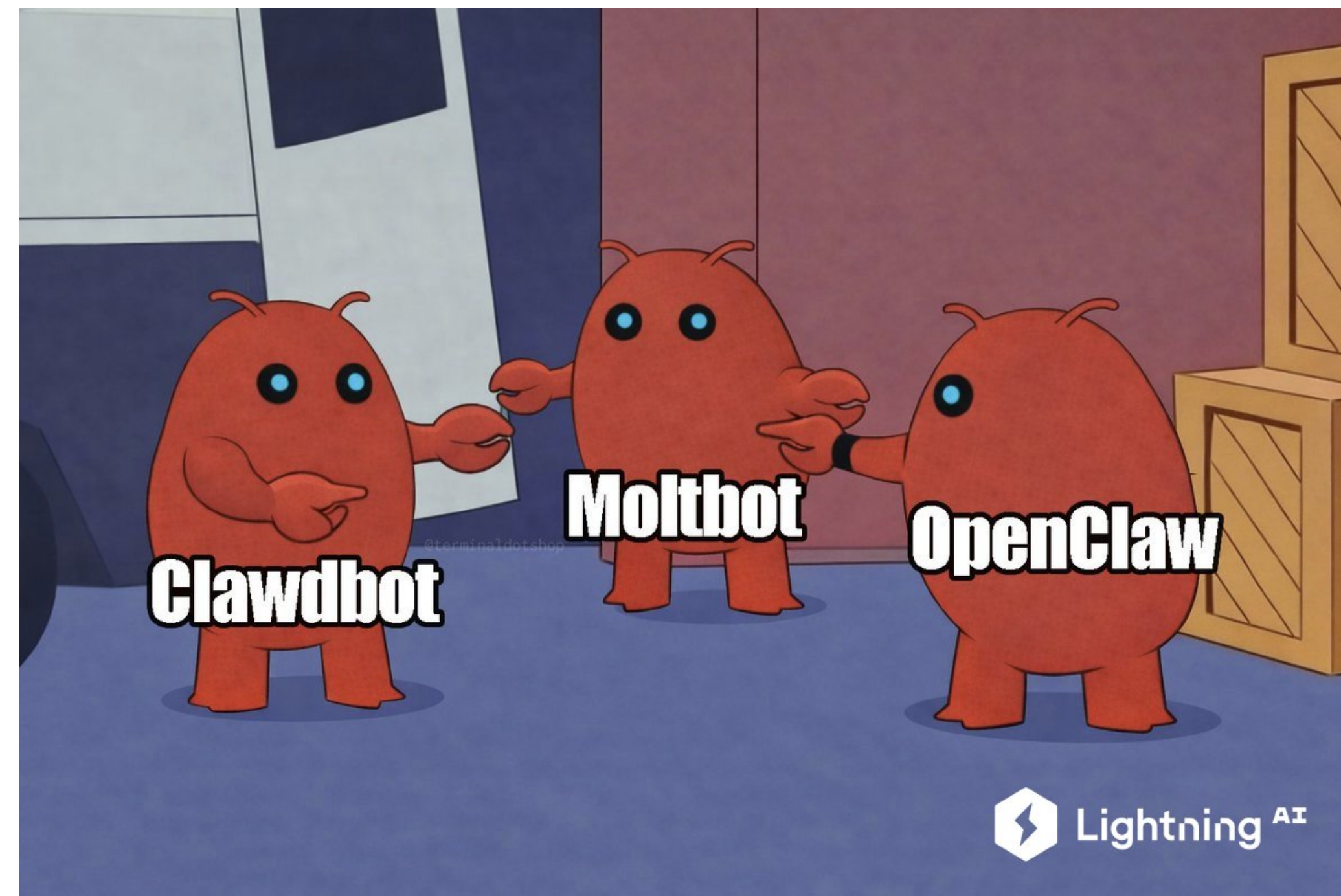


Reinforcement Learning

HSE, winter - spring 2026

Lecture 11: Agentic RL



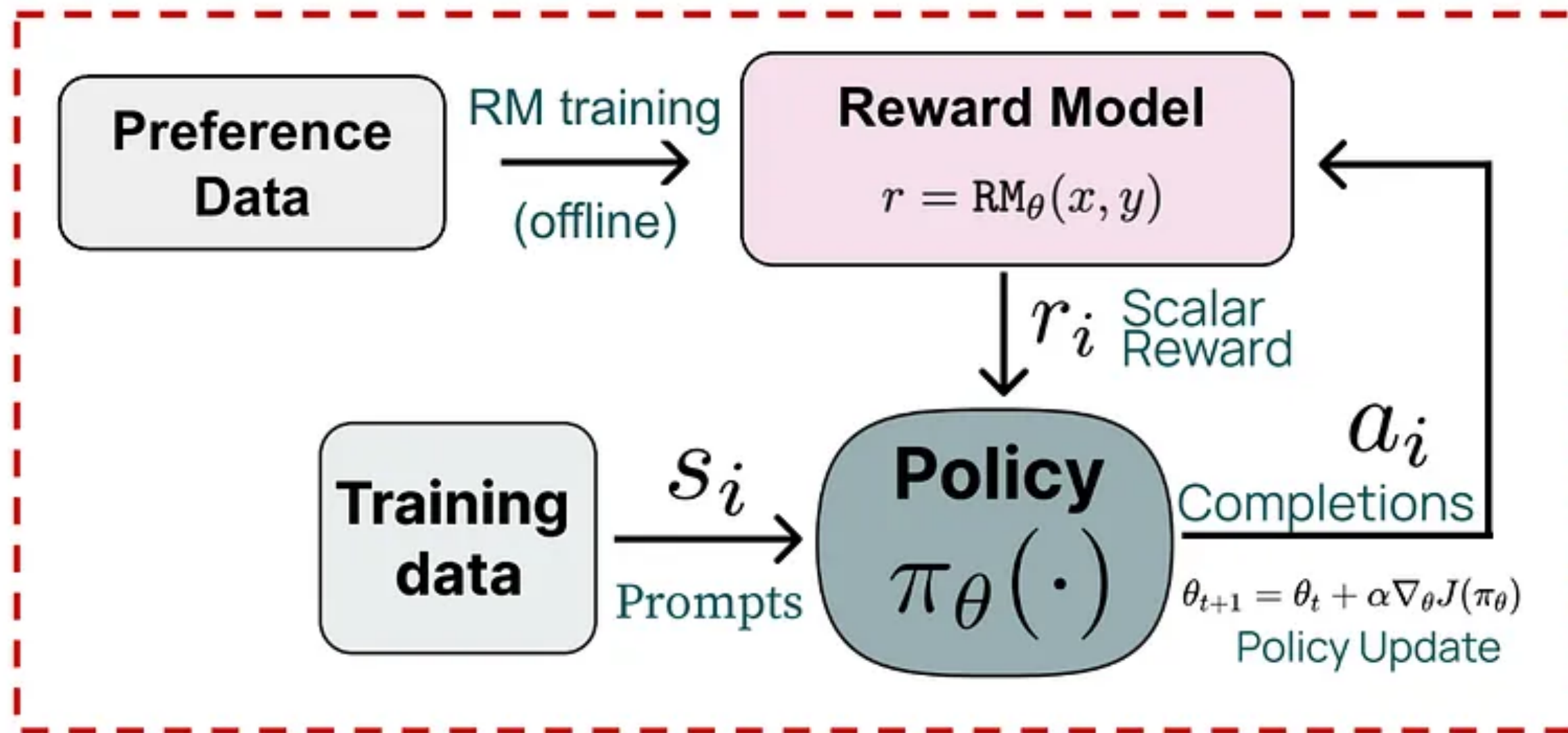
Sergei Laktionov

Source

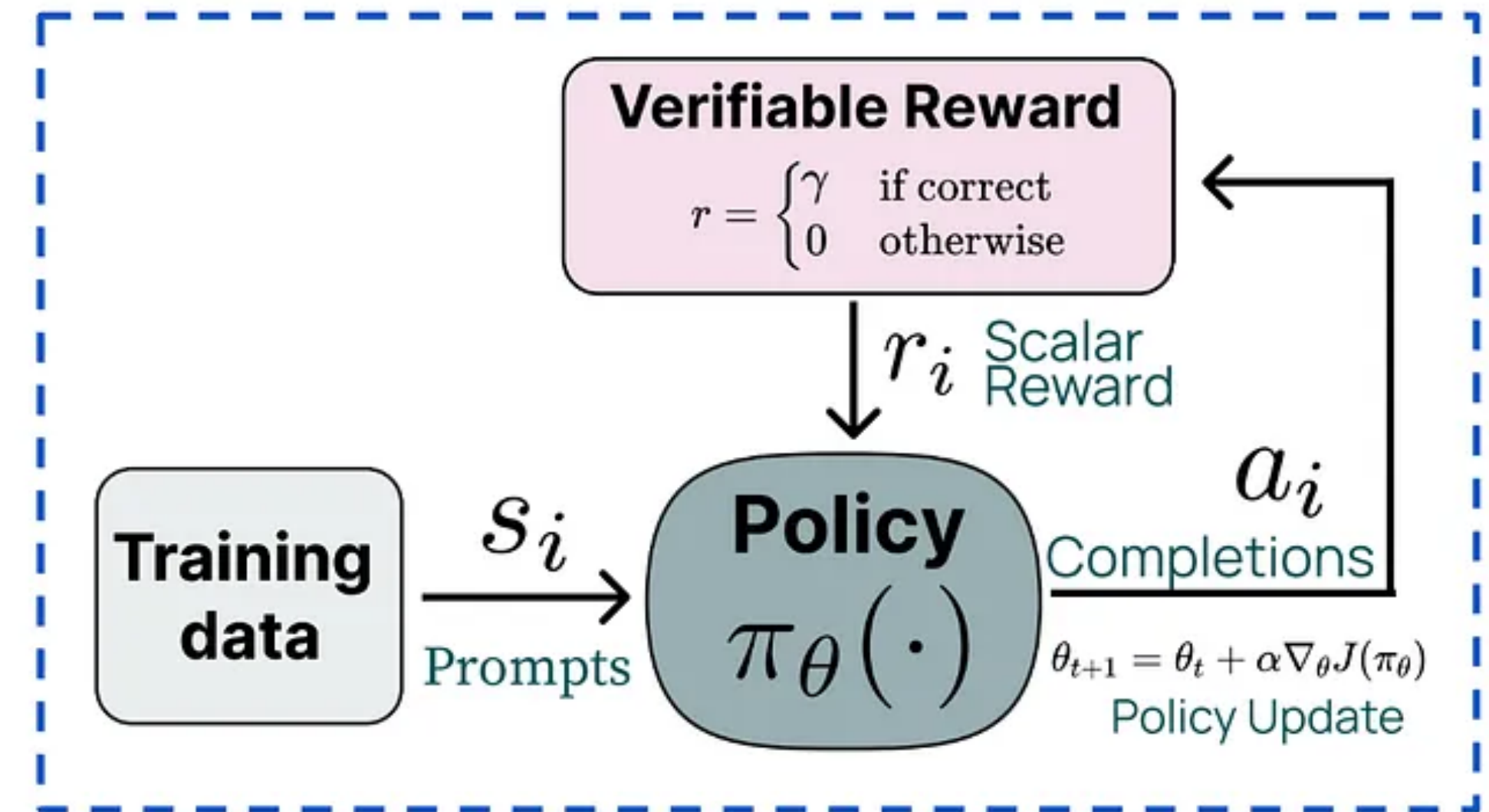
Recap: RLHF and RLVR

Source

Reinforcement Learning from Human Feedback (RLHF)



Reinforcement Learning with Verifiable Rewards (RLVR)

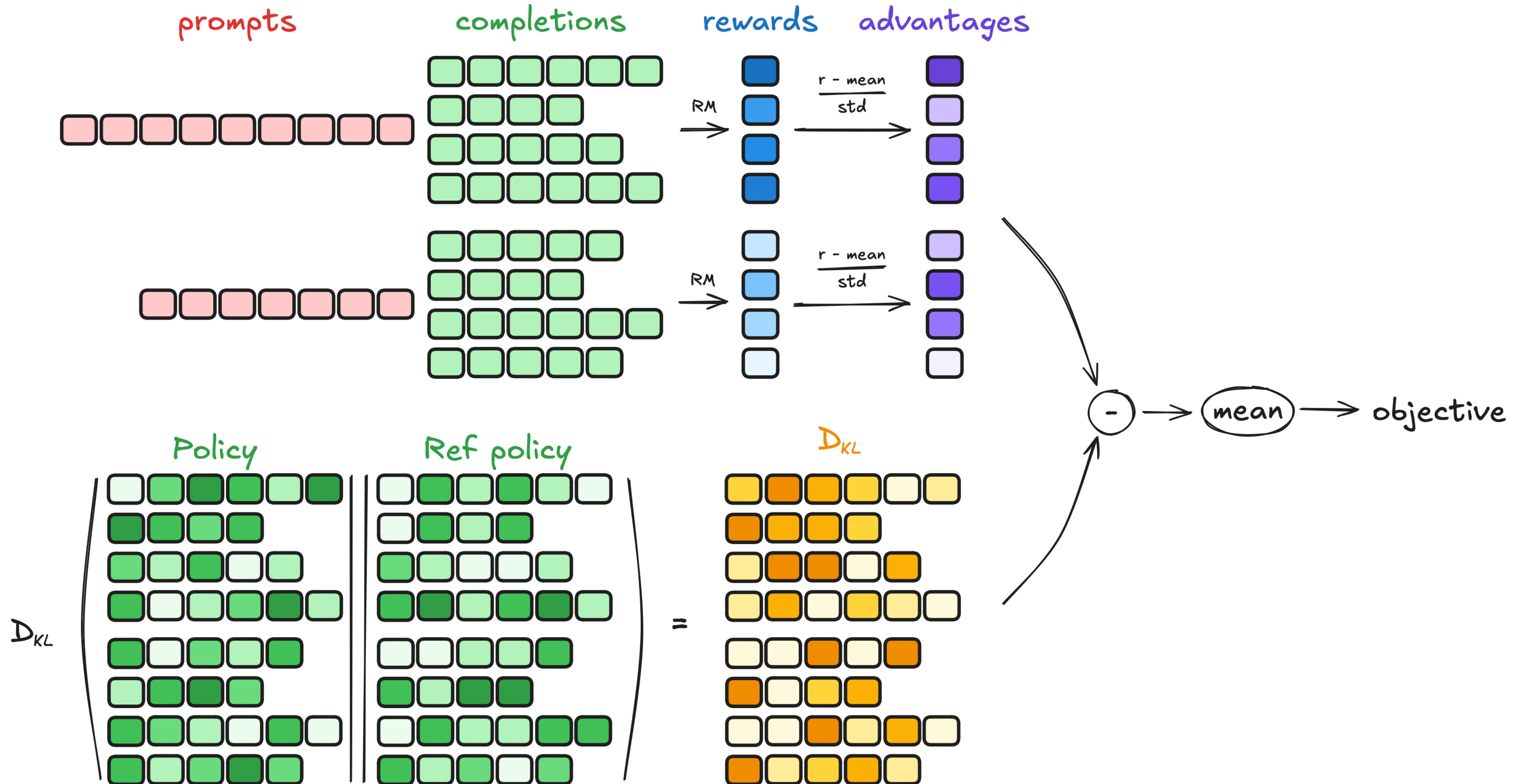


$$\min_{\phi} [-\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \log \sigma(R_{\phi}(x, y_w) - R_{\phi}(x, y_l))]$$

$$\max_{\theta} E_{x \sim \mathcal{D}, y \sim \pi_{\theta}} [R_{\text{verifier}}(x, y)]$$

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}} [R_{\phi}(x, y) - \beta D_{KL}(\pi_{\theta} || \pi_{ref})]$$

Recap: GRPO



Single-Step RL

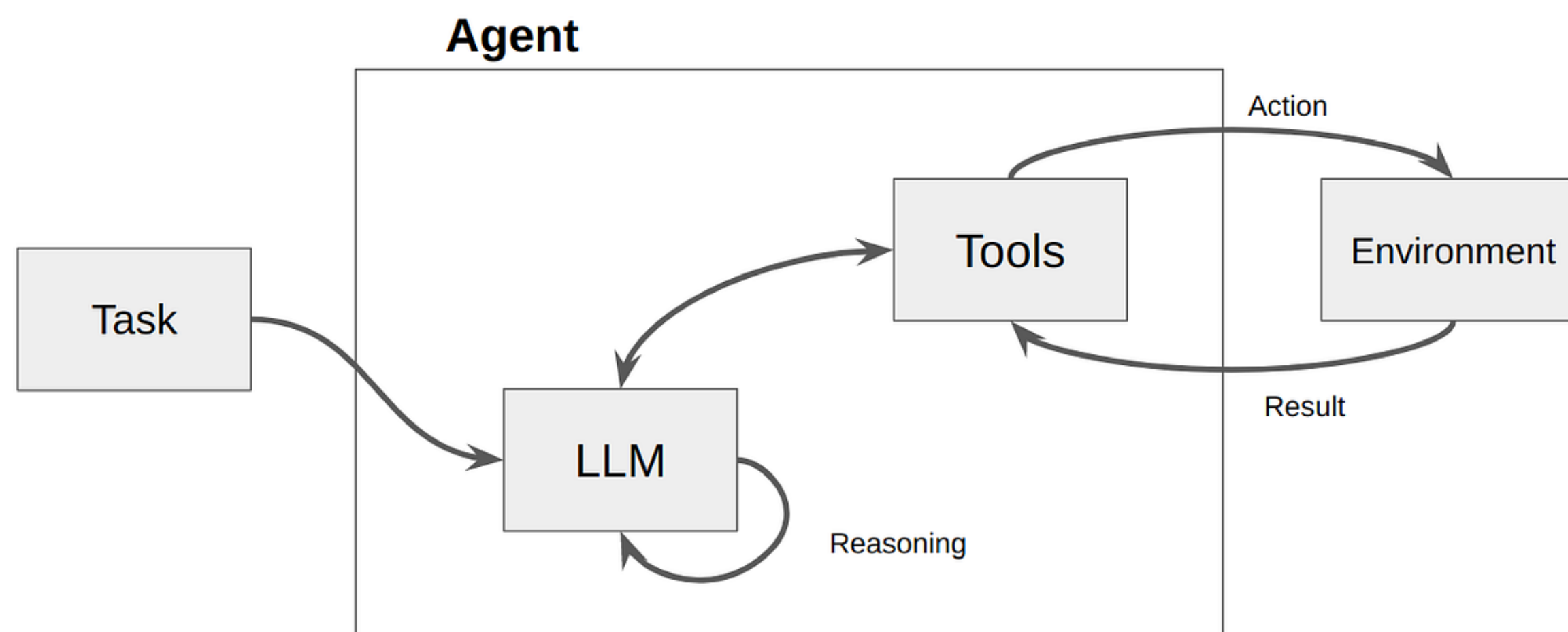
- Vanilla RLVR operates more like an in-context bandit: the model generates a single response, receives an immediate reward and updates.
- There is no temporal depth, no sequential decision-making across states.

Multi-Step RL

- Agent operates in an interactive MDP setting: the model take actions, observe environment feedback, and optimize over extended trajectories with sparse, delayed rewards
- The scope of applications naturally extends beyond closed, verifiable tasks to more complex real-world scenarios, such as travel planning, complex data analysis, software engineering, robotics.

Tool Calling

- The model can call some predefined tools by generating the answer in specific format e.g. JSON or XML
- The answer is parsed and executed
- Some external information is given to the model after tool execution



```
<SYSTEM_PROMPT>
You are ChatGPT, a helpful assistant. Use tools when necessary to provide accurate,
</SYSTEM_PROMPT>

<USER>
What's the weather in London right now?
</USER>

<ASSISTANT_THINKING>
The user is asking for current weather, which is time-sensitive.
I should call the weather widget/tool to retrieve real-time data.
</ASSISTANT_THINKING>

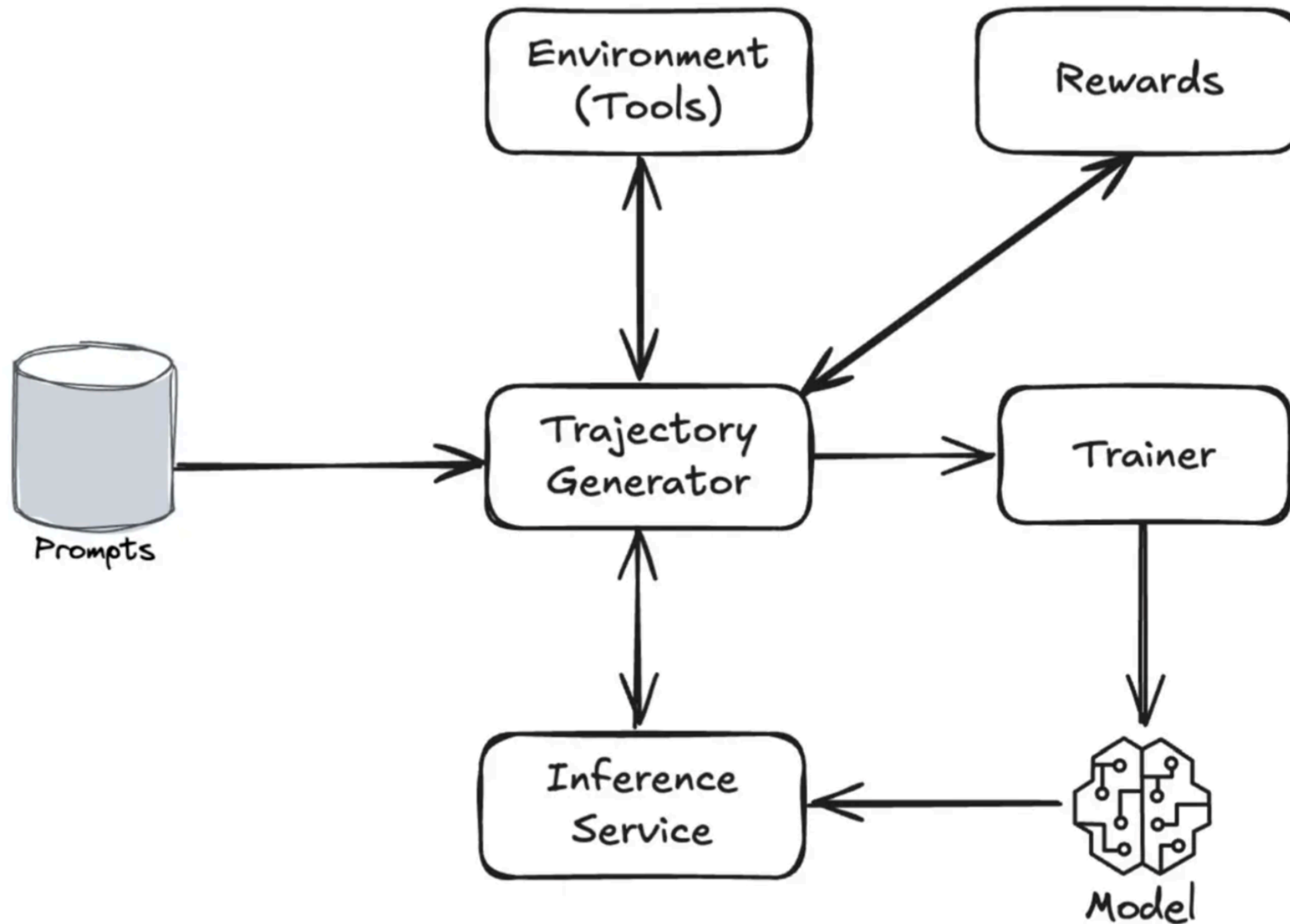
<ASSISTANT_TOOL_CALL>
genui_run|weather_widget_now_with_weather_source|{"location":"London, UK"}
</ASSISTANT_TOOL_CALL>

<TOOL_RESPONSE>
{
  "location": "London, UK",
  "temperature": "12°C",
  "condition": "Partly cloudy",
  "humidity": "68%",
  "wind": "10 km/h"
}
</TOOL_RESPONSE>

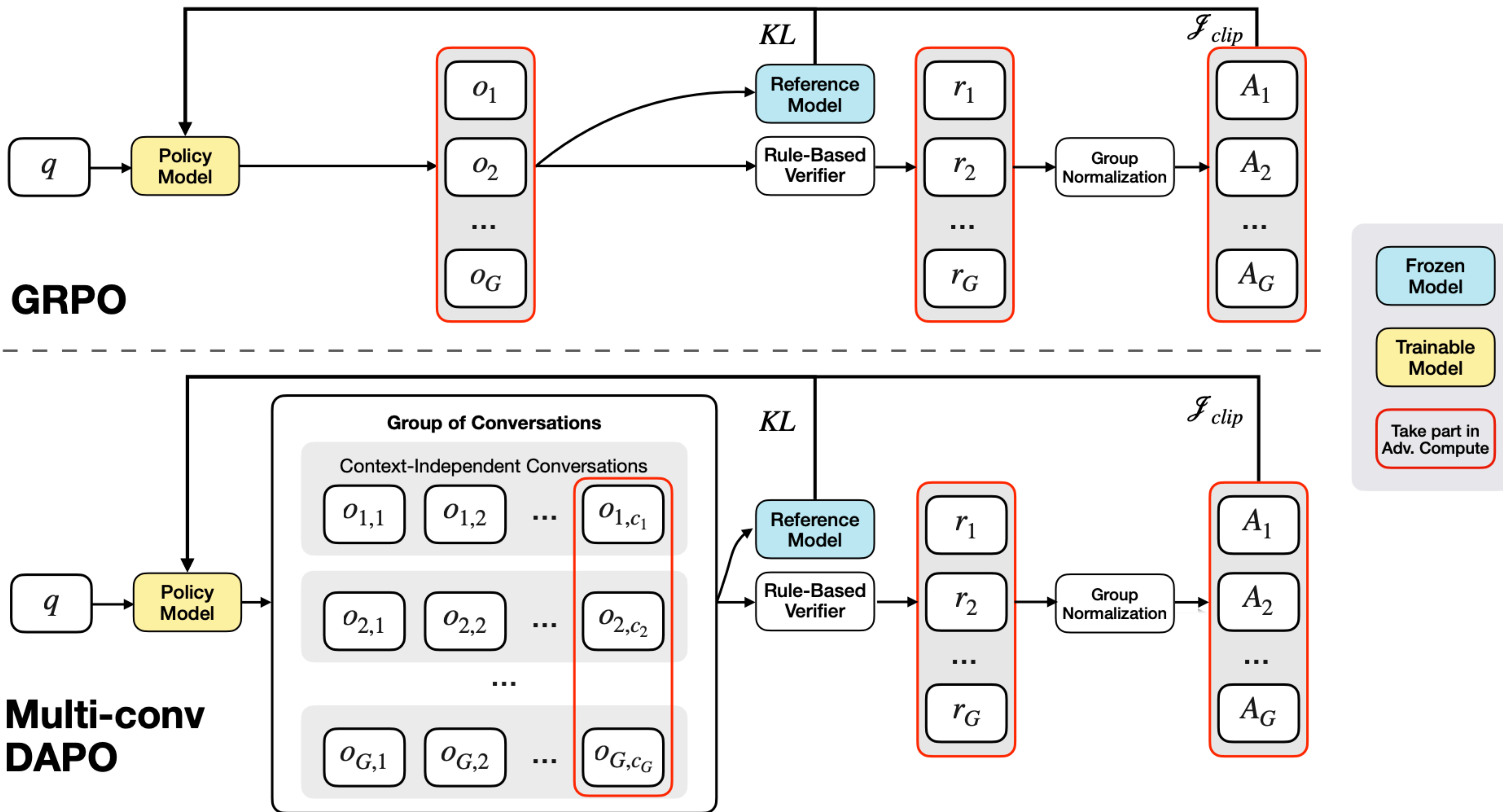
<ASSISTANT_FINAL>
The current weather in London is:

- 🌡️ Temperature: 12°C
- ☁️ Condition: Partly cloudy
- 💧 Humidity: 68%
- 🌬️ Wind: 10 km/h
</ASSISTANT_FINAL>
```

Framework



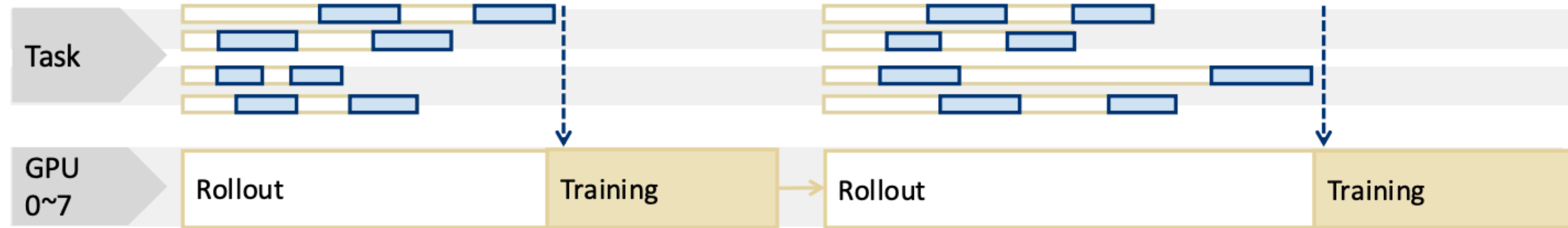
GRPO in Agentic World



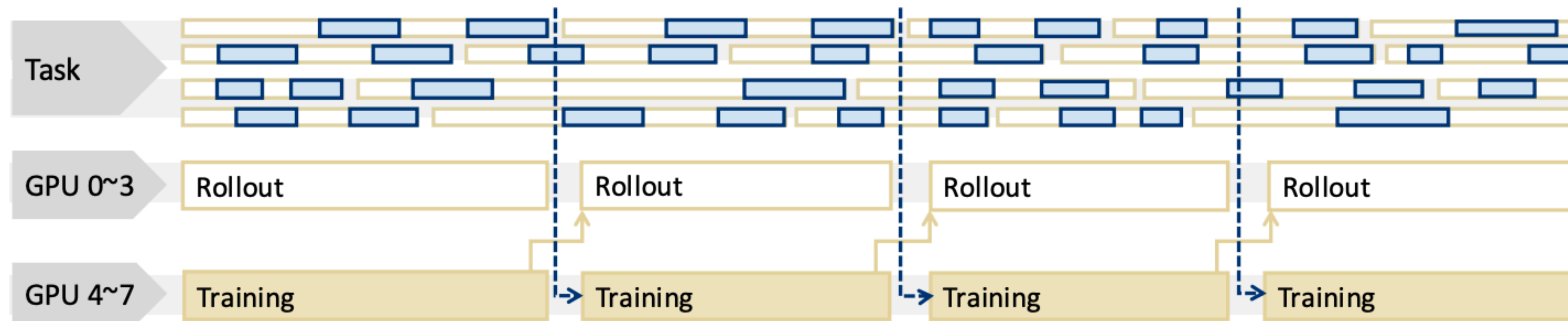
Challenges

1. **Speed:** tasks are heterogeneous, i.e. they differ in complexity and the time required to complete them.
2. **Credit assignment:** outcome-based rewards provide only a single bit of signal that is applied to all tokens in the trajectory.

Sync vs Async Training



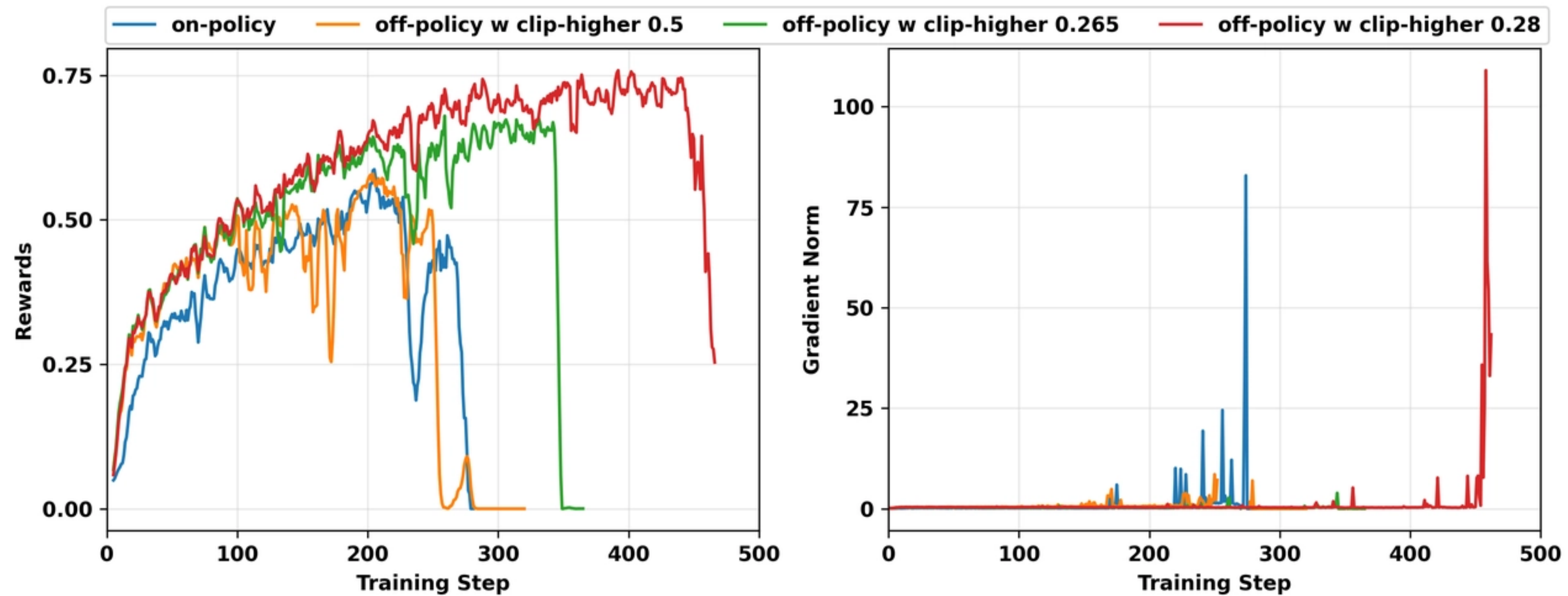
(i) Synchronous Architecture



(ii) Fully Asynchronous Architecture



When Speed Kills Stability



When Speed Kills Stability

In our stack, this mismatch manifested between our vLLM inference sampler and our FSDP trainer. The actual parameter update was:

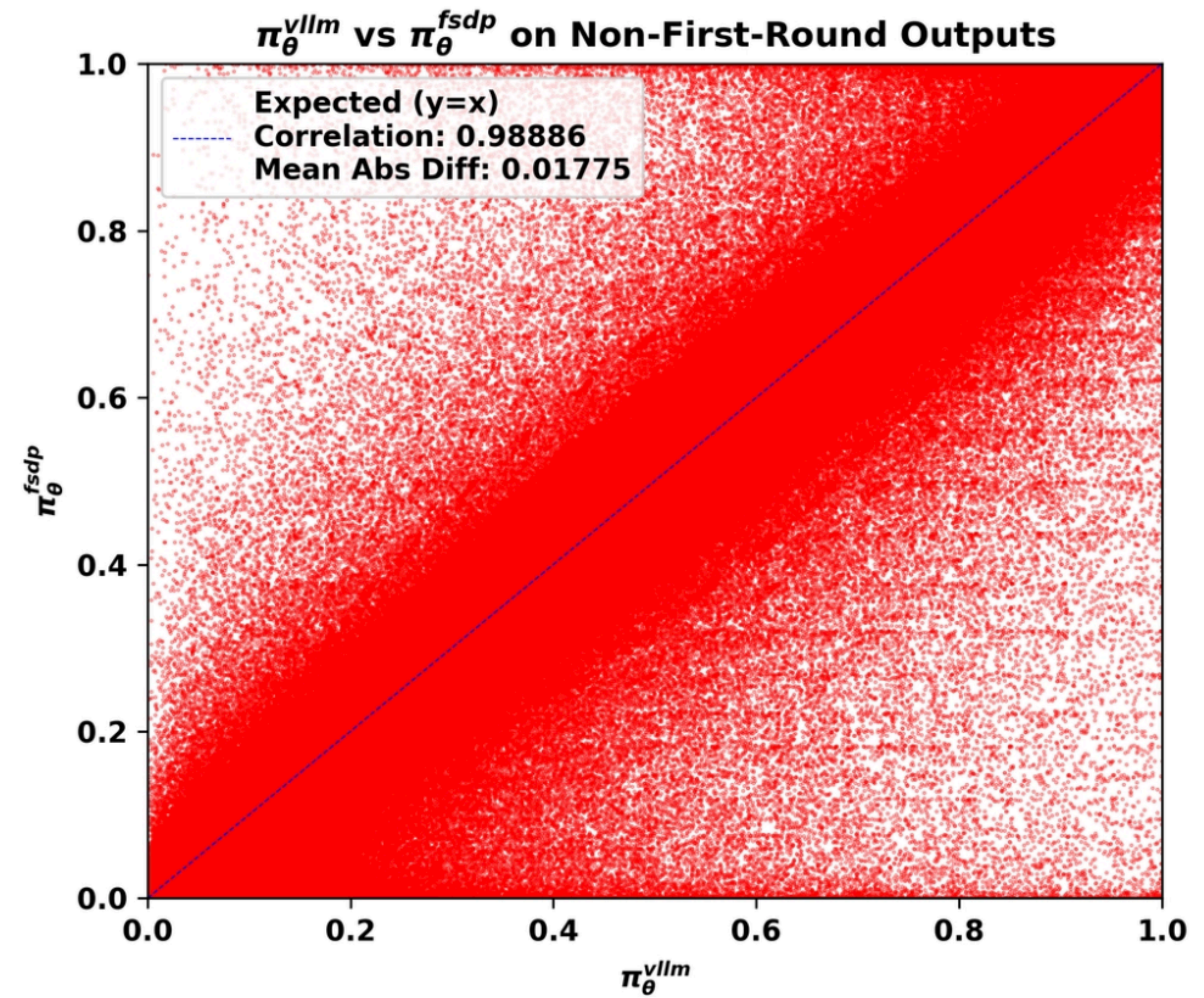
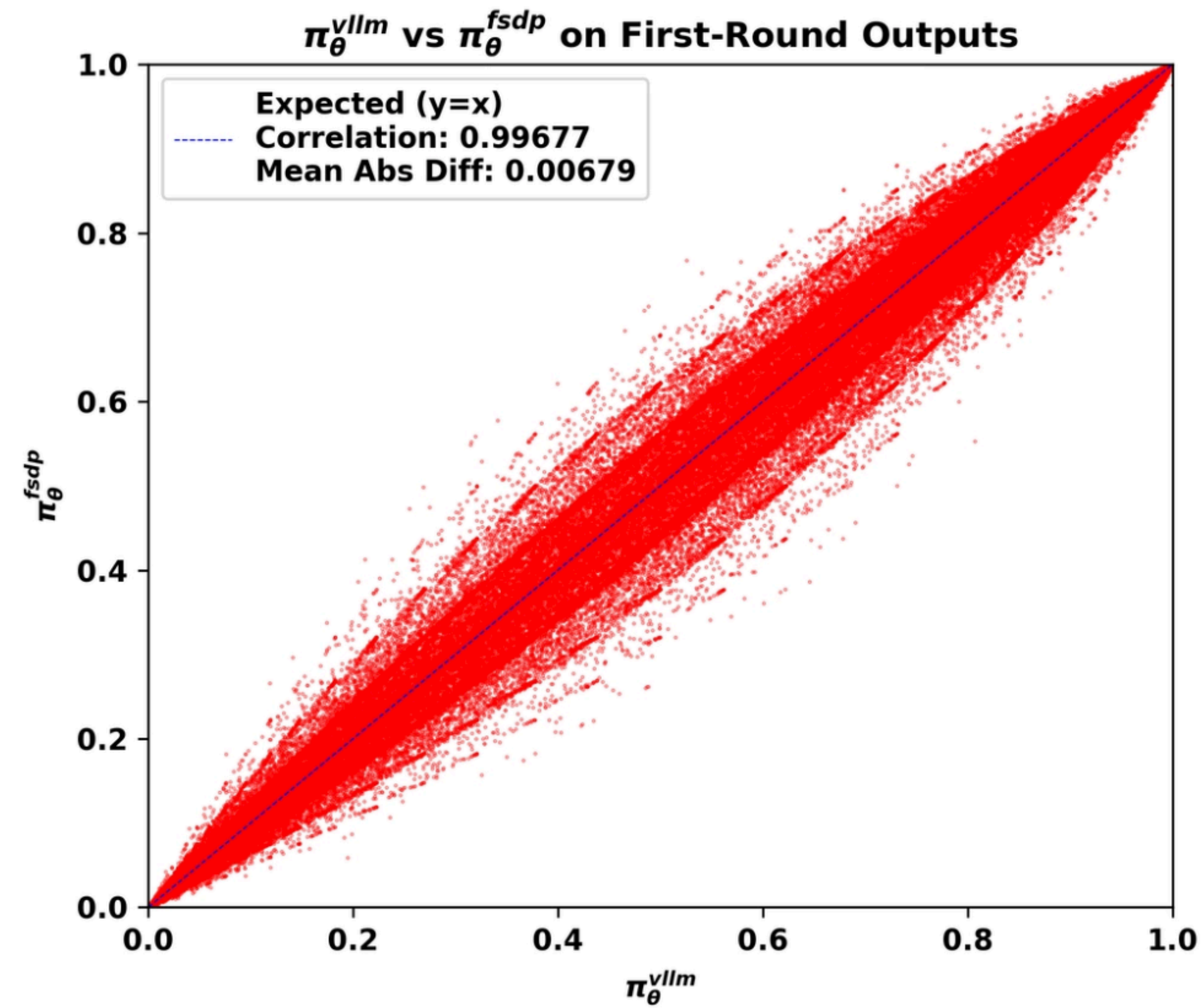
$$\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi_{\theta}^{\text{vllm}}(\cdot|x)} \left[R(x, y) \nabla_{\theta} \log \pi_{\theta}^{\text{fsdp}}(y|x) \right],$$

whereas the theoretical parameter update should be

$$\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi_{\theta}^{\text{fsdp}}(\cdot|x)} \left[R(x, y) \nabla_{\theta} \log \pi_{\theta}^{\text{fsdp}}(y|x) \right].$$

Here x is the prompt sampled from the distribution $\mathcal{D}$, y is the response, R is the reward function, θ is the parameter of LLM, $\pi_{\theta}^{\text{vllm}}$ and $\pi_{\theta}^{\text{fsdp}}$ are the policy implemented in vLLM engine and FSDP engine respectively. To investigate this, we first needed a way to measure it.

When Speed Kills Stability

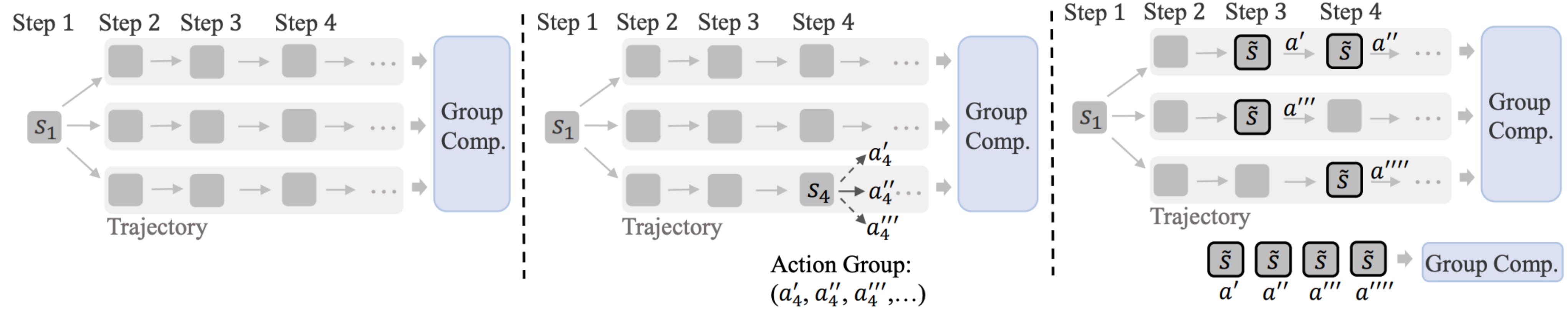


Who is to blame and what to do?

1. The Mismatch Is More Severe in Non-First-Round Outputs
2. More Tool Calls, More Training Instability
3. The Critical Role of Hardware: different GPUs/Kernels
4. The mismatch is non-static and coupled with the training dynamics and model's state.

1. Demystifying RL Collapse from the Training-Inference Mismatch
2. Your Efficient RL Framework Secretly Brings You Off-Policy RL Training

Multi-Turn LLM Agent Training

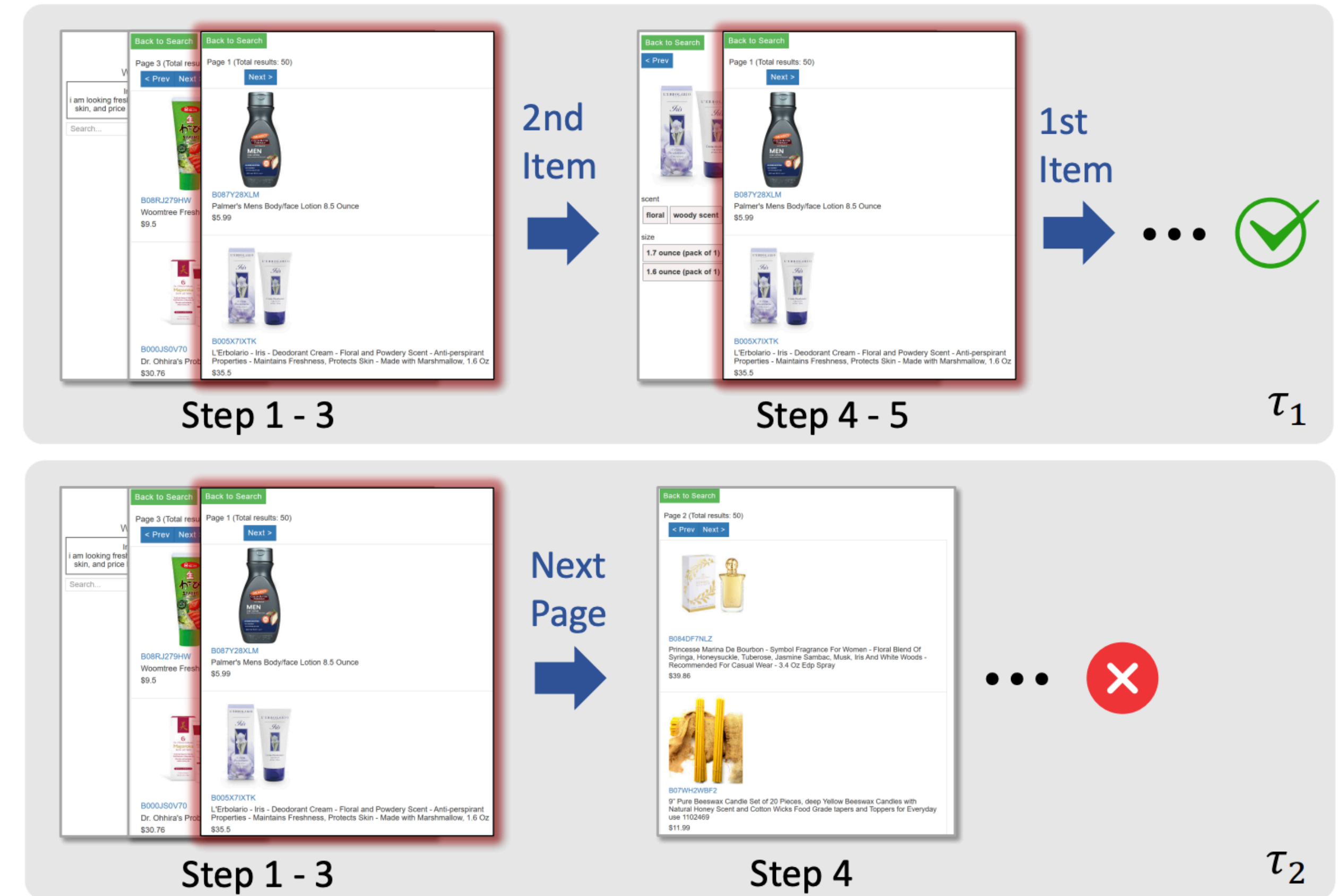


Step-Level Group

$$A^E(\tau_i) = r^{(i)} - \text{avg}(\{r^{(j)}\})$$

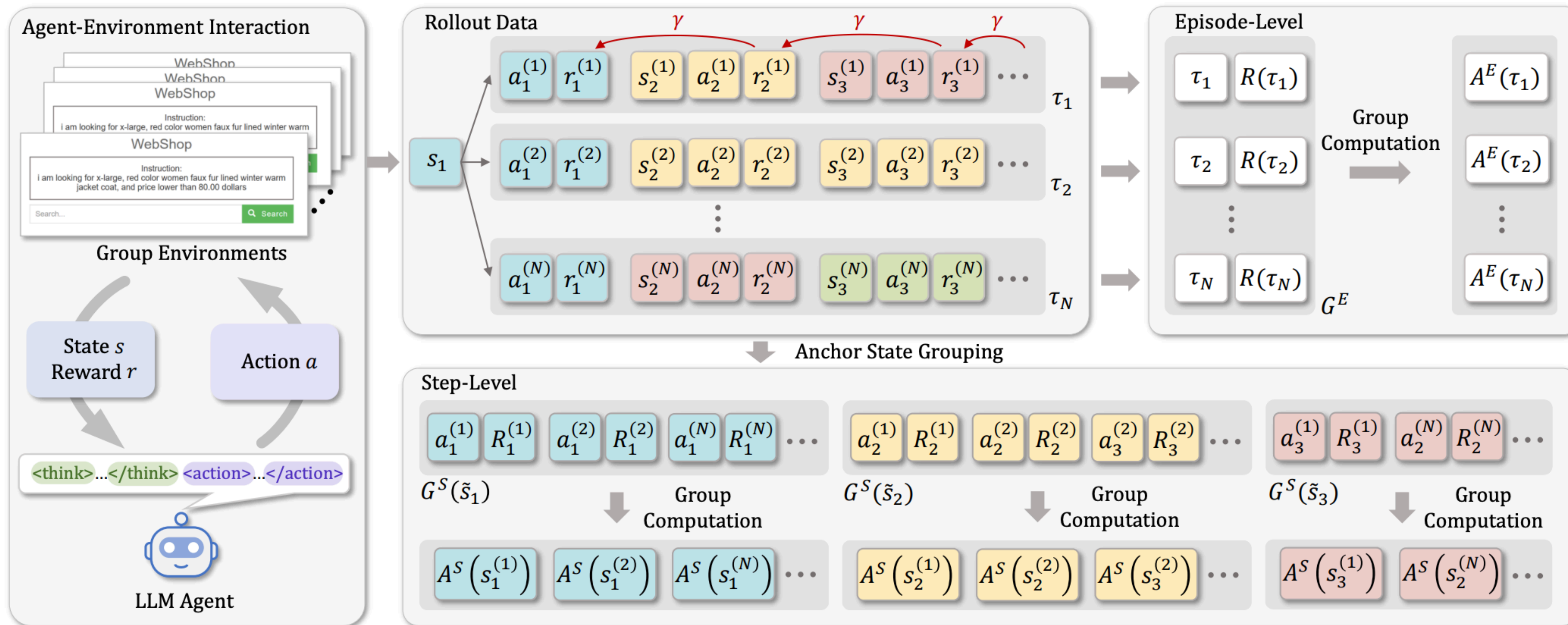
$$A^S(a_t^{(i)}) = r_t^{(i)} - \text{avg}(\{r_t^{(j)} \mid a_t^{(j)} \in G^S\})$$

$$A(a_t^{(i)}) = A^E(\tau^{(i)}) + \omega A^S(a_t^{(i)})$$



$$A^S(\text{1st Item}) > A^S(\text{2nd Item}) > A^S(\text{Next Page})$$

GiGPO



Background

1. Small Language Models are the Future of Agentic AI
2. AgentRL: Scaling Agentic Reinforcement Learning with a Multi-Turn, Multi-Task Framework
3. MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent
4. Group-in-Group Policy Optimization for LLM Agent Training
5. The Bitter Lesson Behind Building Agentic RL in Terminal Environments
6. OpenClaw-RL: Train any agent simply by talking
7. How Claude Code is built

Thank you for your attention!